AI Lab assignment 13

Tejas Patil
U24AI061

Q)

```python
class Symbol:
    def __init__(self, name):
        self.name = name
        self.value = None

def NOT(p):
    return not p
def AND(p, q):
    return p and q
def OR(p, q):
    return p or q
def IMPLIES(p, q):
    return (not p) or q
def IFF(p, q):
    return p == q

def evaluate_expr(expr):
    expr = expr.strip()

    if expr.startswith("(") and expr.endswith(")"):
        count = 0
        is_wrapped = True
        for i in range(len(expr) - 1):
            if expr[i] == "(": count += 1
            elif expr[i] == ")": count -= 1
            if count == 0:
                is_wrapped = False
                break
        if is_wrapped:
            return evaluate_expr(expr[1:-1])

    if expr == "P": return P.value
    if expr == "Q": return Q.value
```

```
    if expr == "R": return R.value

    words = expr.split()

    for op in ["IFF", "IMPLIES", "OR", "AND"]:
        count = 0
        for i in range(len(words) - 1, -1, -1):
            word = words[i]
            if ")" in word: count += word.count(")")
            if "(" in word: count -= word.count("(")

            if count == 0 and word == op:
                left = " ".join(words[:i])
                right = " ".join(words[i+1:])

                if op == "AND": return AND(evaluate_expr(left),
evaluate_expr(right))
                if op == "OR": return OR(evaluate_expr(left),
evaluate_expr(right))
                if op == "IMPLIES": return IMPLIES(evaluate_expr(left),
evaluate_expr(right))
                if op == "IFF": return IFF(evaluate_expr(left),
evaluate_expr(right))

    if expr.startswith("NOT"):
        return NOT(evaluate_expr(expr[4:]))

    raise ValueError("Invalid Expression")

def truth_table_2(P, Q, expr):
    print("\nExpression:", expr)
    print("P Q Result")
    for p_val in [True, False]:
        for q_val in [True, False]:
            P.value, Q.value = p_val, q_val
            result = evaluate_expr(expr)
            print('T' if p_val else 'F', 'T' if q_val else 'F', 'T' if
result else 'F')

def truth_table_3(P, Q, R, expr):
```

```
    print("\nExpression:", expr)
    print("P Q R Result")
    for p_val in [True, False]:
        for q_val in [True, False]:
            for r_val in [True, False]:
                P.value, Q.value, R.value = p_val, q_val, r_val
                result = evaluate_expr(expr)
                print('T' if p_val else 'F', 'T' if q_val else 'F', 'T' if
r_val else 'F', 'T' if result else 'F')

P = Symbol('P')
Q = Symbol('Q')
R = Symbol('R')

truth_table_2(P, Q, "NOT P IMPLIES Q")
truth_table_2(P, Q, "NOT P AND NOT Q")
truth_table_2(P, Q, "NOT P OR NOT Q")
truth_table_2(P, Q, "NOT P IMPLIES NOT Q")
truth_table_2(P, Q, "NOT P IFF NOT Q")
truth_table_2(P, Q, "(P OR Q) AND (NOT P IMPLIES Q)")

truth_table_3(P, Q, R, "(P OR Q) IMPLIES NOT R")
truth_table_3(P, Q, R, "((P OR Q) IMPLIES NOT R) IFF ((NOT P AND NOT Q)
IMPLIES NOT R)")
truth_table_3(P, Q, R, "((P IMPLIES Q) AND (Q IMPLIES R)) IMPLIES (Q
IMPLIES R)")
truth_table_3(P, Q, R, "(P IMPLIES (Q OR R)) IMPLIES (NOT P AND NOT Q AND
NOT R)")
```